

Storage Engines for PageIndex Retrieval: A Workload-Grounded Benchmark of MongoDB, PostgreSQL, DuckDB, and SQLite

Junyao Dong

Abstract. PageIndex turns documents into large hierarchical JSON trees that an agent navigates from the root to the relevant nodes. We benchmark four storage engines (MongoDB, PostgreSQL/JSONB, DuckDB, and SQLite, ConDB’s current engine) on the exact operations ConDB’s retrievers issue against storage, at scales up to ten million nodes. Across navigation reads, content fetch, writes, concurrency, and storage footprint, MongoDB is competitive on every operation except one: fetching a large subtree (`get_subtree`), a materialized-path range scan matching tens of thousands of nodes. The initial benchmark recorded a multi-second P95 there, but a controlled re-measurement—in isolation and under concurrent load, with the collection fully resident in cache—finds a stable P95 of roughly 0.4s, about $3\times$ PostgreSQL rather than the order of magnitude the first run suggested; the seconds-level figure was a contention artifact of the concurrent multi-engine run, not a steady-state cost. We trace the remaining gap to MongoDB’s document-at-a-time read model: each matched node is a separate `FETCH`, so the cost tracks the number of nodes in the subtree. We also measure fixes on Community Edition (Section 6). In the measured resident-cache regime, because the cost is the *number* of fetches, only removing them helps, and only modestly—a covering index buys $\sim 1.4\times$ —since the sub-second baseline leaves little to gain. That ranking does not replace the production-scale schema rule: when many trees make the hot working set larger than RAM, split `text` first so `structure/title/summary` can stay cached.

1 Introduction

ConDB indexes a document as a PageIndex tree: a hierarchy of nodes carrying a title, a summary, page indices, and (at the leaves) text. Retrieval is a top-down traversal: the retriever starts at the root, lists a node’s children to decide where to descend, pulls a subtree for the language model to read, and fetches the content of the nodes it selects. The project question is whether a document store (MongoDB) is the right home for this data and, if so, what to optimize. This report answers both with measurements, including a measured optimization plan for the one weak spot (Section 6).

We deliberately benchmark only the operations ConDB actually issues. Generic database operations that the retrieval path never performs (ancestor-to-root walks, page-range filters, naive substring scans) are excluded, because their results would not inform the optimization.

2 Workload and methodology

Operations. The retrievers (`beam_retriever.py`, `block_retriever.py`) reach storage through four calls (Table 1). Everything measured below is one of these, plus the storage footprint, the write path (incremental index and reasoning-trace edits), and concurrency.

Table 1: The storage operations ConDB retrieval issues.

Operation	Storage call	Used for
Point lookup	<code>get_node(tree_id, node_id)</code>	Navigate to / read one node
Expand children	<code>get_children(tree_id, node_id)</code>	List children, pick where to descend
Get subtree	<code>get_subtree(tree_id, node_id, depth)</code>	Render the tree view, pull a block
Fetch content	<code>get_entity(tree_id, node_id)</code>	Read selected nodes' content

Data. Synthetic PageIndex trees in the canonical format, validated against ConDB's `DocumentTreeAdapter`, at two scales (Table 2). Content is random words; what the benchmark exercises is the tree shape and the per-node payload sizes.

Table 2: Datasets.

Dataset	Nodes	JSON size	Depth	Fan-out
Medium	70,843	85 MB	5	6–12
Large	10,000,000	14.06 GB	8	6–14

Engines and setup. MongoDB 7.0 (Community Edition) and PostgreSQL 16 (JSONB document column) run as servers in Docker over localhost; DuckDB and SQLite are embedded. All engines run on the same 96-core Linux host with 1 TB of RAM. Every engine stores one record per node with identical logical fields and the indexes each query needs. The tree is flattened to one row per node; `get_subtree` uses a materialized path with a byte-order range predicate, identical across engines (PostgreSQL's `path` column uses `COLLATE "C"` so its range matches the others' binary comparison). The subtree query returns node ids; adding the title and summary the tree view also renders would cost the relational engines more but not MongoDB, whose non-covering `FETCH` already reads the whole document, so adding fields to the projection costs it nothing—making this choice conservative toward MongoDB. Each read is measured over 500 sampled nodes (200 subtree roots, drawn at depth ≤ 3), fixed by seed and identical across engines; every query was checked to return the same row count on all four engines. Each concurrency level runs its client processes for five seconds. The host has ample memory, so every engine's data is cache-resident and reads are not disk-bound.

Fairness caveats. SQLite and DuckDB run in-process, so a query is a function call; MongoDB and PostgreSQL pay per-call client-server overhead (a localhost round trip plus protocol handling) that the embedded engines never incur. The fair single-call comparison is therefore MongoDB versus PostgreSQL; absolute microseconds across the embedded/server line are not comparable, and the concurrency section (independent processes, no GIL) shows real server throughput. MongoDB's `storageSize` is post-compression (WiredTiger snappy); the others are uncompressed on-disk files, so MongoDB's uncompressed logical size is reported separately, and the random-word content likely compresses better than real prose, flattering the compressed figure. All measurements are single-host. The operations through concurrency use one tree per database; multi-tenancy, batched reads, and deletion are measured separately below on 25 co-resident trees.

3 Results

3.1 Storage and ingest

Table 3: Storage and ingest (large dataset). On-disk totals include the indexes. MongoDB holds 15.21 GB of logical BSON in 5.27 GB of data on disk (WiredTiger snappy).

Engine	On-disk	Index	Uncompr.	Ingest (n/s)	Build
DuckDB	5.02 GB	0	–	348,831	10 s
MongoDB	5.81 GB	536 MB	15.21 GB	41,387	69 s
PostgreSQL (JSONB)	17.71 GB	1.19 GB	–	62,816	52 s
SQLite	19.51 GB	980 MB	–	52,880	38 s

DuckDB and MongoDB are the most compact (Table 3), DuckDB through columnar encoding and MongoDB through compression; even uncompressed, MongoDB’s 15.21 GB of BSON is no larger than the relational engines’ raw tables. PostgreSQL and SQLite are several times larger on disk.

3.2 Retrieval operations

The navigation reads (point lookup, children, content fetch) are sub-millisecond on every engine but DuckDB, which is not built for OLTP point reads. MongoDB runs roughly 3× behind PostgreSQL per call on these reads (Table 4), part of it per-call client overhead, but at well under a millisecond everywhere the gap carries no practical cost; the optimization space is not here. Only `get_subtree` costs at scale: a large subtree returns tens of thousands of documents. The initial run (Table 4) put MongoDB’s P95 in the seconds while the others stayed under ~150ms, but that figure did not hold up under scrutiny—a controlled re-measurement (Section 5) finds a stable ~0.4s. The materialized-path range also beats a recursive `parent_id` traversal, so ConDB should keep it.

Table 4: Retrieval read latency (ms, lower is better). Best per row in bold. The large `get_subtree` P95 (2478) is from the original multi-engine run; a controlled re-measurement finds ~0.4s (Section 5).

Operation	MongoDB	PostgreSQL	DuckDB	SQLite
<i>Medium, P50</i>				
Point lookup (<code>get_node</code>)	0.238	0.081	2.705	0.008
Expand children (<code>get_children</code>)	0.251	0.090	0.856	0.017
Get subtree (<code>get_subtree</code>)	0.498	0.210	2.700	0.091
Fetch content (<code>get_entity</code>)	0.228	0.071	2.750	0.007
<i>Large, P50 / P95</i>				
Point lookup	0.270 / 0.32	0.087 / 0.09	5.59 / 8.97	0.012 / 0.014
Expand children	0.282 / 0.32	0.099 / 0.14	4.28 / 5.05	0.023 / 0.032
Get subtree (~36k rows)	30.9 / 2478	10.7 / 123	9.9 / 47	11.8 / 135
Fetch content	0.249 / 0.32	0.081 / 0.09	5.21 / 8.35	0.012 / 0.013

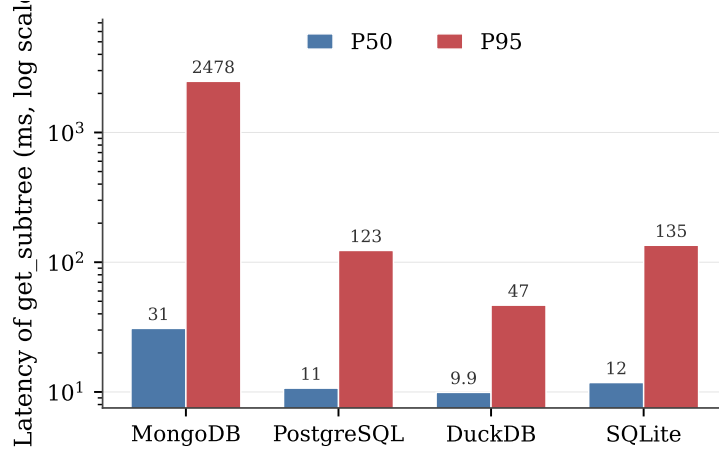


Figure 1: `get_subtree` latency on the large dataset (log scale). The medians are within a small factor. MongoDB’s P95 outlier at 2.5s is from this multi-engine run; in isolation and under concurrent load it is ~ 0.4 s (Section 5).

3.3 Write path

On the write path (Table 5), PostgreSQL posts the lower update latency of the two servers, but each `jsonb_set` versions the row and leaves a dead tuple for a later `VACUUM`. WiredTiger is MVCC too, yet it reclaims obsolete document versions automatically, so an update-heavy workload carries no growing bloat; its measured on-disk delta is small but noisy, since WiredTiger reports storage in checkpoint-sized chunks. DuckDB is unsuitable for OLTP writes.

Table 5: Write path (medium): 5,000 field updates, 2,000 incremental inserts. “Bloat” is on-disk growth from the updates.

Engine	Upd. P50	Upd. ops/s	Insert ops/s	Bloat
MongoDB	0.242	4,050	5,586	0 MB
PostgreSQL (JSONB)	0.123	7,856	8,732	7 MB
DuckDB	2.966	332	450	7 MB
SQLite	0.017	40,442	21,815	0 MB

3.4 Concurrency

Single-call latency understates the server engines. Driving point lookups from a growing pool of independent OS processes (no GIL bottleneck), both server engines gain throughput with added clients, and the gap to the embedded engines narrows far below what the single-call numbers suggest (Figure 2). MongoDB rises steadily across the whole range; PostgreSQL peaks near 16 clients; SQLite is fastest in absolute terms but peaks earliest, within the first few clients, then degrades as the processes contend on its single file. Repeated runs move the peak positions and heights somewhat; the shapes hold.

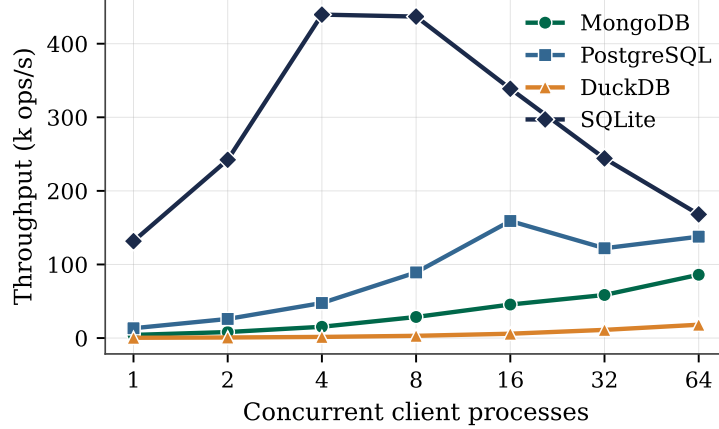


Figure 2: Point-lookup throughput versus concurrent client processes (medium).

3.5 Multi-tenancy, batched reads, and deletion

Mapping the retriever’s calls to storage surfaces three operations the sections above leave out: a per-query tenant filter when many trees share one store, the batched id-list read the navigation step issues, and whole-tree deletion. We measure them on the medium tree replicated into 25 co-resident trees (1,771,075 rows in one store), with every read filtered by `tree_id` and served by a compound index led by `tree_id`.

Multi-tenant selectivity. With 25 trees in one collection, a `tree_id`-led index holds each read close to its single-tree latency (Table 6): the cost tracks index depth, not the number of co-resident trees. MongoDB rises about $1.5\times$ on point lookup and $2.6\times$ on subtree over the single-tree medium figures (Table 4) for a $25\times$ larger store, with no scan blow-up, and the MongoDB–PostgreSQL ordering is unchanged.

Table 6: Per-query reads with 25 trees co-resident in one store, each filtered by `tree_id` (P50, ms).

Operation	MongoDB	PostgreSQL	DuckDB	SQLite
Point lookup	0.345	0.124	3.772	0.008
Expand children	0.372	0.147	3.073	0.014
Get subtree	1.273	0.312	4.605	0.069

Batched reads. The navigation step returns a list of candidate node ids, so the retriever fetches several nodes at once. One batched read (`$in` on MongoDB, `IN` on the relational engines) beats the point-lookup loop it replaces on every engine, and the margin grows with batch size because each iteration of the loop on a server engine is a round trip. At 200 ids MongoDB is 4.6 ms batched against 273 ms looped, a $60\times$ gap; PostgreSQL is 1.4 ms against 29 ms (Table 7). MongoDB gains the most because batching amortizes the per-call client overhead that costs it on single reads; in-process SQLite, with no round trip, gains about $2\times$. The retriever should resolve a candidate set with one batched read, not a loop.

Table 7: Batched id-list read at 10, 50, and 200 ids against the 200-id point-lookup loop it replaces (P50, ms per batch).

Engine	Batched (n ids)			Loop
	10	50	200	200
MongoDB	0.46	1.56	4.56	273
PostgreSQL (JSONB)	0.27	0.78	1.40	29
DuckDB	19.6	22.2	39.1	790
SQLite	0.03	0.13	0.50	0.97

Deletion. Removing a tree (one tenant, 70,843 of 1,771,075 rows) is a lifecycle operation, not part of retrieval. The delete is fast on every engine, under four seconds, but none returns disk space on the delete alone (Table 8). SQLite (`VACUUM`) and PostgreSQL (`VACUUM FULL`) reclaim the removed tree’s footprint through a full, locking rewrite that takes 10 to 17 s; MongoDB `compact` and DuckDB do not measurably shrink the file for a single-tenant deletion, which is WiredTiger’s documented behavior of not returning space to the operating system on delete. For MongoDB this mirrors the write path: WiredTiger carries no vacuum debt under updates, and in exchange a one-off bulk delete does not auto-shrink. WiredTiger reports `storageSize` in checkpoint-sized chunks, so changes below tens of megabytes are reporting granularity, not reclaimed space.

Table 8: Whole-tree delete and space reclamation (delete one of 25 trees).

Engine	Delete (rows/s)	Reclaim	Reclaim time	On-disk (before → after)
PostgreSQL (JSONB)	157,578	<code>VACUUM FULL</code>	17.3 s	2.77 → 2.66 GB
SQLite	141,842	<code>VACUUM</code>	10.6 s	2.56 → 2.46 GB
MongoDB	60,680	<code>compact</code>	0.1 s	911 MB, no shrink
DuckDB	20,171	<code>checkpoint</code>	–	790 MB, no shrink

None of the three shifts the report’s conclusion: MongoDB holds the tenant filter at scale, wins the batched read outright, and on deletion trails only on a locking-free shrink that retrieval never triggers.

4 Why MongoDB trails on `get_subtree`

The cause is structural, not a configuration mistake. MongoDB’s unit of read is the whole document. A secondary index (here on `path`) is a separate B-tree holding only the indexed field and a record id; to return any other field, the engine must *fetch* the full BSON document and apply the projection to it. A point lookup is one such fetch; a large subtree is tens of thousands, the biggest in the workload far more. The data is cache-resident (and pages in the WiredTiger cache are stored uncompressed), so the cost is neither disk I/O nor decompression: the time goes to per-document work. Because the `path` index is non-covering, each matched record id triggers a `FETCH` of the whole BSON document—`text` payload and all, even though the query needs only a node id—and tens of thousands of those fetches batch through the cursor. Multiplied by subtree size, that per-document overhead is the cost of the operation. The other operations touch few documents, so they stay fast.

The cost therefore scales with the number of nodes in the subtree: the largest subtrees, near the root, are the slowest. How large that tail actually is—and whether the original run’s multi-second

figure is a steady-state property or an artifact of that run’s contention—is the subject of the next section.

The row and columnar engines do less work per node. A row store visits each matching row but materializes only the requested column; the wide `text` sits unread in the row (PostgreSQL does not even decompress the JSONB datum, since `node_id` is a separate column). A column store reads only the requested column in vectorized batches. Both could go further still with a covering index that answers the range from the index alone (Section 6). All of these do far less per-row work than a per-document fetch.

This is the price of the document model, and it is also why MongoDB wins elsewhere: storing all of a node’s fields together makes “fetch one whole entity” fast (point and children reads), in-place field updates cheap (the write path), the layout shard-friendly (concurrency), and the on-disk form compact (storage). MongoDB is optimized for a different access pattern, and the PageIndex workload plays to it on every operation but one.

5 Re-measuring the tail

The original `get_subtree` figure (P95 2478 ms) was recorded while the other three engines, the write benchmark, and the concurrency sweep ran on the same host. To separate the query’s own cost from that contention, we re-measured it on the ten-million-node tree under controlled conditions: one measuring client, the collection freshly loaded and fully resident, the same 200 seeded subtree paths.

Re-measured in isolation, P95 is $\sim 0.30\text{--}0.42$ s (P50 $\sim 30\text{--}40$ ms), stable across repeated runs—an order of magnitude below the first run. Driving the same query while 16, 32, then 64 concurrent processes hammer point lookups at the same `mongod` barely moves it (Table 9): P95 rises from 331 to 376 ms across the sweep, $1.1\times$, never approaching seconds. Throughout, the WiredTiger cache holds the whole collection (~ 17 GB resident in a 540 GB cache) and reports zero application-thread evictions—so the tail is neither eviction nor, as it turns out, ordinary read contention.

Table 9: `get_subtree` re-measured on the 10M tree, one measuring client, while N background processes drive concurrent point lookups (id-only projection, ms).

Concurrent load N	P50	P95	P99	WT evictions
0	31.4	331	476	0
16	39.4	357	555	0
32	44.1	415	534	0
64	39.0	376	627	0

The corrected picture is milder than the first run implied. Steady-state, `get_subtree` on the large tree is ~ 0.4 s at P95 and $\sim 30\text{--}40$ ms at the median—roughly $3\times$ PostgreSQL at *both* (its 123 ms tail and 11 ms median), not the $\sim 20\times$ the original P95 alone suggested. The 2478 ms was a property of the loaded machine during the first run—most likely whole-system resource contention, or a cold-cache first touch before the collection was warm—not of the query in steady state. The mechanism from the previous section still holds (each matched node is a separate `FETCH`, so cost grows with subtree size), but its steady-state magnitude is sub-second.

6 Optimizing `get_subtree`

Because the harness was already in place, we did not stop at recommending fixes—we ran them on the ten-million-node tree and measured (Table 10). Each keeps MongoDB on Community Edition with no engine change. The short version for this measurement is deliberately narrow: with the collection fully resident in cache, the document model’s cost is the *number* of fetches, so only removing fetches helps, and even that is modest because the steady-state baseline is already sub-second.

Table 10: `get_subtree` fixes measured on the 10M tree (one client; view returns `node_id+title+summary`, `id` returns `node_id` only; ms).

Variant	P50	P95	Result
Baseline, view (<code>text</code> inline)	39.6	419	reference
Subset Pattern (split <code>text</code>), view	39.2	419	no change
Lean covering <code>{path,node_id}</code> , <code>id</code>	19.3	191	$\sim 1.4\times$, covered
Fat covering <code>+title,summary</code> , view	33.5	350	$\sim 1.2\times$; 4.66 GB index
Lean covering + batched <code>\$in</code> , view	137	1314	$\sim 3\times$ slower

Table 10 is therefore a *resident-cache* optimization ranking, not a full deployment ranking. In a large deployment with many trees or tenants, the constraint can reverse: the working set may exceed RAM, and then the first-order question is which bytes remain cache-resident. Under that condition, splitting `text` out of the hot node collection is the first schema requirement, followed by `tree_id`-led indexes/sharding so each query stays tenant-local, and then by lean covering indexes or denormalized subtree views. The experiment below does not quantify that cache-constrained benefit; it only shows that the benefit cannot appear when both schemas already fit in cache.

6.1 What the measurements say

Splitting the text field does nothing in this resident-cache experiment. The intuitive fix—move the large `text` into a separate collection so the scanned documents are small—was the leading hypothesis, and it is wrong here. It shrinks the collection by $\sim 75\%$ ($4.97 \rightarrow 1.25$ GB) yet leaves `get_subtree` P95 unchanged ($419 \rightarrow 419$ ms). The reason is the cache: at 540 GB it holds the whole collection many times over (Section 5), so document *size* never gates the scan—document *count* does. Each of the $\sim 36k$ matched nodes is a separate `FETCH`, and shrinking each one does not reduce their number. The Subset Pattern’s premise—a working set too large for RAM—does not hold on this host. For production scale, this is exactly the boundary condition to watch: if inline `text` pushes the structure/title/summary working set out of cache, the split becomes a cache-residency requirement even though it did not move this resident-cache latency number.

A covering index is the only real lever, and a modest one. Removing the `FETCH` is the one thing that helps. A lean `{path,node_id}` index serves the `id`-only subtree query index-only (`explain` confirms `PROJECTION_COVERED`, `totalDocsExamined`: 0) for $\sim 1.4\times$. But the tree view also needs `title` and `summary`: extending the index to cover them works ($\sim 1.2\times$) yet inflates it to 4.66 GB—as large as the collection’s own data—because `summary` is a multi-kilobyte key, confirming

the fat-key concern. Resolving the view with a covered id scan plus a batched `$in` is worse, not better ($\sim 3\times$ slower): a $\sim 36\text{k-id}$ `$in` overflows the 16 MB command limit (forcing ~ 36 chunked round trips) and re-fetches the same documents anyway. Keep the anchored materialized-path range as the predicate—an unanchored regex cannot form a range and must scan the whole index—and treat cache sizing and `zstd` as irrelevant here: the data is already resident, so neither touches the per-fetch cost.

A larger win needs structural denormalization, untested here. Nested Sets or a tree-order `_id` on a clustered collection turns the subtree into one range query or a sequential read; a pre-computed subtree document turns it into a point read. All are viable precisely because the tree is write-once (their cost is paid once at ingest), and all remove fetches wholesale rather than shrinking them. We did not measure them: with the baseline already at $\sim 0.4\text{s}$, the open question is whether the schema complexity is worth shaving a sub-second, infrequent operation. (`$graphLookup` is not a candidate—it re-traverses `parent_id`, still materializes documents, and in 7.0 is capped at 100 MB and ignores `allowDiskUse`.)

6.2 What will not help

A newer MongoDB version does not close the gap. Upgrading the engine does not fix this without a schema change. Against this operation—a projection-heavy range scan over ordinary documents—the 8.0–8.3 line offers nothing: the 8.0 *Express* path is a point/equality fast path on a single index (`_id`-style, no range, sort, or skip), 8.0 *block processing* is scoped to time-series collections, and the `$graphLookup` disk-spill fix (SERVER-23980) lands only in 8.2.0 (reverted from 8.1) and only helps the recursive alternative anyway. The document-at-a-time `FETCH` is unchanged across the line.

What Community Edition does not offer. The feature that would fix this most cleanly—a column-store index, so a projection-heavy scan reads only the projected column, as the relational engines do—is Atlas-only. But `PageIndex` never scans the whole corpus, so a column store solves a problem this workload does not have; the schema fixes above address the one it does. Self-managed Search and Vector Search run on Community since 8.2 (a separate `mongot` process, public preview, replica set required) but serve relevance retrieval, not a range-projection scan. No Atlas- or Enterprise-only feature addresses the document-at-a-time read model itself, so the only levers are schema, index, and storage configuration—all on Community.

6.3 What remains open

Three things the measurements did not pin down. First, the exact origin of the original 2478 ms—whole-system contention during the multi-engine run, or a cold-cache first touch before the collection was warm; point-lookup contention alone does not reproduce it (Section 5), so the cause is heavier than the concurrency the workload itself generates. Second, the Subset Pattern under a cache-constrained host, where the `inline-text` collection spills RAM but the `structure/title/summary` collection fits; that is the production-scale premise this resident-cache experiment could not exercise. Third, the measured payoff of the structural-denormalization options above. None changes the measured conclusion: `get_subtree` is a sub-second, $\sim 3\times$ -PostgreSQL operation whose only cheap resident-cache fix—a covering index—has limited upside.

7 Is MongoDB the wrong tool?

The introduction asked whether a document store is the right home for this data, and `get_subtree` is where the question bites: a relational or columnar engine does this operation natively, reading one column and never touching the `text` blob, which is why PostgreSQL posts a ~ 123 ms tail to MongoDB’s ~ 0.4 s (Section 5). On the bare operation the relational model is the better fit, and this report does not pretend otherwise.

But the gap is $\sim 3\times$ on a sub-second, infrequent operation—not the order of magnitude the first run implied—and it sits against MongoDB’s wins on every other operation the workload issues: point, children, and content reads; vacuum-free updates; concurrency; footprint. Three workload facts keep it in proportion. The subtree view needs only `node_id`, `title`, and `summary`, never the large `text`. The tree is largely write-once after ingest. And the largest subtrees sit near the root, which a top-down traversal pulls at most once or twice per query before descending into progressively smaller, faster ones. The verdict is conditional but, for *this* workload, clear: a $\sim 3\times$ tail on the one projection-heavy scan does not outweigh MongoDB’s advantages elsewhere.

8 Conclusion

For the PageIndex retrieval workload, MongoDB is competitive on point and children reads and content fetch, accumulates no dead-tuple bloat under updates (no vacuum debt), has a compact footprint, and scales monotonically under concurrency with the lowest tail latency of the server engines (though not their highest absolute throughput). Its one relative weakness is `get_subtree`: a controlled re-measurement puts its steady-state P95 at ~ 0.4 s, roughly $3\times$ PostgreSQL and $\sim 3\times$ at the median too—not the multi-second tail the first run recorded, which we trace to contention rather than to the query. The cost is the per-node `FETCH` count, which tracks the number of nodes in the subtree. The practical reading splits by regime. In the measured resident-cache regime, `get_subtree` is the one place MongoDB trails, the gap is $\sim 3\times$ on a sub-second, infrequent operation, and not a reason against MongoDB for this workload; the only cheap lever is a covering index, worth just $\sim 1.4\times$ (Section 6). In a production-scale, cache-constrained deployment, the priority order reverses: split `text` first so the hot structure/title/summary working set can remain cached, keep every index and shard key led by `tree_id`, and then consider covering indexes or denormalized subtree views. That production ordering is a schema requirement to protect cache residency, not a latency gain quantified by this resident-cache experiment.